

Trabalho Prático 2

Sistema de Escalonamento Logístico

Júlia Almeida Luna (202401424)
julialuna@ufmg.br

1. Introdução

Este trabalho prático tem como objetivo solucionar uma demanda do sistema logístico dos Armazéns Hanoi. O problema modelado envolve o roteamento e acompanhamento de pacotes em uma rede de armazéns. Cada pacote percorre um trajeto definido desde sua origem até o seu destino enfrentando limitações como a capacidade de transporte entre armazéns, latência, intervalos de envio e tempo de remoção de pacotes armazenados.

Além disso, uma particularidade do sistema é a adoção da lógica LIFO (Last-In First-Out) nas seções de armazenamento dos armazéns, o que impõe restrições adicionais à ordem de retirada dos pacotes.

A solução proposta consiste na construção de uma simulação detalhada utilizando tipos abstratos de dados (TADs) para representar pacotes, armazéns, transportes e o escalonador de eventos, a fim de reproduzir o comportamento do sistema real acompanhando eventos como armazenamentos, remoções, transportes e entregas.

2. Método

O projeto foi implementado em C++ utilizando a programação orientada a objetos. Os principais módulos implementados consistem em TADs para Pacotes, Armazém e o Escalonador que implementa todo o processo de simulação através da criação de eventos. Essas estruturas principais utilizam diversas estruturas de dados clássicas que garantem um desempenho adequado e uma organização modular e robusta do código.

2.1. Estruturas de Dados

Destaca-se a utilização das seguintes estruturas:

- *MinHeap*: esta estrutura é utilizada no TAD Escalonador para gerenciar a execução de eventos do sistema em ordem cronológica através das chaves deles. O *MinHeap* garante um gerenciamento eficiente, uma vez que retorna o evento que deve ser executado em $O(\log(n))$
- Pilhas: utilizada no TAD Armazém para representar cada seção que ele possui. Permite fazer uma simulação do armazenamento e recuperação de pacotes mais próxima da realidade, que respeita a lógica LIFO (através do uso de uma pilha auxiliar) e permite a implementação da lógica do custo de retirada.
- Grafo: utilizado para representar a topologia da rede de armazéns, definindo transportes possíveis entre armazéns. A estrutura em lista de adjacência, contribui diretamente para a definição das seções internas de cada armazém, uma vez que cada seção corresponde a um armazém vizinho.
- Fila: esta estrutura é utilizada durante a busca em largura feita no grafo que define as rotas de cada pacote considerando o armazém de origem, o de destino e os transportes possíveis.
- Lista: estrutura utilizada no armazenamento da rota de cada pacote, o grande benefício na sua utilização consiste na flexibilidade de seu tamanho, que permite o

armazenamento de rotas com diferentes comprimentos de forma eficiente, sem desperdício de memória.

2.2. TAD Pacote

O TAD Pacote tem como responsabilidades principais o armazenamento de dados úteis como a hora de postagem, o id do armazém de origem e destino e a rota que ele terá que percorrer para chegar no armazém de origem para o destino. O cálculo desta rota é de responsabilidade do próprio TAD Pacote que a partir do grafo da topologia de armazéns define a rota que leva o pacote seu armazém de origem ao de destino, utilizando o algoritmo de busca em largura, isto é feito na função *'calculate_route'*.

É válido destacar também outra responsabilidade do TAD que é o armazenamento do estado do pacote que podem ser 5: 1 - Não foi postado, 2 - Chegada escalonada em um armazém, 3 - Armazenado em uma seção associada ao armazém do próximo destino, 4 - Removido da seção para transporte e 5 - Entregue. A atualização dos estados é feita pela função *'set_state'*. Esses estados são fundamentais no processo de simulação, uma vez que permitem a garantia da execução do evento correto no pacote e a verificação da entrega de todos os pacotes.

2.3. TAD Armazém

O TAD armazém tem como uma de suas funções a definição de quantas e quais seções o armazém terá a partir da lista de adjacência do vértice correspondente ao armazém, esta ação é executada pela função *'define_sections'*. Ademais, todo o manuseamento de retirada e armazenamento de pacotes em seções demandada pelo escalonador é executado pelo armazém através das funções *'store_package'* e *'retrieve_package'*.

2.4. TAD Escalonador

Por fim, o TAD Escalonador é o componente que realmente conduz toda a simulação de transporte logístico. Para isso, ele assume diversas responsabilidades secundárias, entre as quais se destaca a criação e o gerenciamento de eventos. Cada evento, seja de transporte ou de chegada de pacote, recebe uma chave de 13 dígitos, cujas duas variações são mostradas na Figura 1. Essas chaves são essenciais porque é por meio delas que o Escalonador compara os eventos e decide qual deve ser executado em seguida.

Dígito	1	2	3	4	5	6	7	8	9	10	11	12	13
Pacote	Tempo						Pacote					Tipo	
Transporte	Tempo						Origem		Destino			Tipo	

Figura 1: Especificação das chaves dos eventos

A simulação ocorre na função *'simulate_deliveries'* por meio da criação e do processamento sequencial de dois tipos de eventos, transporte e chegada de pacotes, até que todos os pacotes atinjam seus destinos finais. No início, agendam-se, no *MinHeap* de eventos, tanto os eventos

de chegada de cada pacote em seu armazém de origem quanto todos os possíveis eventos de transporte entre pares de armazéns. A cada iteração, o escalonador extrai do *MinHeap* o evento de menor chave e o executa, garantindo que os eventos sejam processados na ordem correta e conduzindo a simulação até a entrega de todos os pacotes.

Na execução dos eventos de transporte, feita através da função ‘*execute_transport_events*’, o Escalonador retira da seção apropriada do armazém de origem todos os pacotes destinados a um armazém de destino específico. Se o número de pacotes ultrapassar a capacidade do transporte, os que sobram são armazenados novamente na mesma seção. Em seguida, para cada pacote liberado, ele agenda no *MinHeap* de eventos um evento de chegada, que representará o instante em que o pacote chega ao próximo armazém.

Quando um evento de chegada é executado, através da função ‘*execute_package_events*’, o Escalonador verifica se aquele armazém é o destino final do pacote. Se for, atualiza o estado do pacote para “entregue”. Caso contrário, armazena o pacote na seção correspondente ao seu próximo destino dentro daquele mesmo armazém. Com essa sequência de criação e execução de eventos, o Escalonador garante a continuidade da simulação até que todos os pacotes tenham sido devidamente entregues.

3. Análise de Complexidade

Nesta seção, será apresentada a análise de complexidade não apenas do mecanismo principal de simulação de eventos (com seus custos de transporte, chegada e operações em *MinHeap*), mas também das funções auxiliares fundamentais que inicializam os TADs Armazém e Pacote que garantem o funcionamento do sistema.

3.1. Definição das seções nos armazéns

Durante a inicialização do sistema, ao ler o grafo que modela a topologia dos armazéns, o TAD Armazém invoca o método ‘*define_sections*’ para alocar um vetor de pilhas (uma para cada seção) e o vetor ‘*index_section_mapping*’, que armazena o ID do armazém vizinho correspondente a cada índice de pilha.

Para preencher esse mapeamento, percorre-se toda a lista de adjacência do armazém, resultando em custo de tempo $O(n)$, onde n é o número de vizinhos. A memória empregada cresce linearmente com o número de seções, caracterizando uma complexidade espacial $O(n)$.

3.2. Cálculo das rotas dos pacotes

Durante a leitura das informações dos pacotes, como hora de postagem e armazém de origem e destino, calculamos a rota de cada pacote usando busca em largura sobre o grafo da topologia dos armazéns. Esse procedimento de *Breadth-First Search* percorre cada vértice e cada aresta do grafo exatamente uma vez, resultando em complexidade de tempo $O(V+E)$, em que V é o número de armazéns (vértices) e E o número de ligações (arestas). Em termos de memória, a função aloca três vetores de tamanho V (visitados, predecessores e pilha de reconstrução do caminho), além da fila de vértices em processamento, totalizando complexidade espacial $O(V)$.

3.3. Simulação de eventos

3.3.1. Complexidade de tempo

Para estabelecer a complexidade de tempo e espaço da função '*simulate_deliveries*', devemos considerar seu cenário mais adverso: uma topologia em forma de árvore completamente desbalanceada, em que o percurso mais longo vai da raiz até a folha, totalizando $n-1$ arestas para n armazéns, este será o pior caso. Nessa mesma configuração, todos os pacotes são postados simultaneamente e seguem exatamente essa única rota, gerando máxima contenção e obrigando o simulador a processar o maior número possível de eventos de transporte e chegada ao longo do percurso.

A simulação de eventos vai contar com a execução de eventos de transporte e de chegada para executar toda a simulação. Desse modo, devemos inicialmente determinar a complexidade dessas funções e depois quantas vezes elas serão executadas.

- **Execução dos eventos de chegada**

Para determinar a complexidade do núcleo do Sistema de Escalonamento Logístico, o simulador de eventos discretos, precisamos avaliar separadamente o custo de cada tipo de evento. Nesta subseção, examinamos a função '*execute_arrival_event*', que, ao processar um evento de chegada, simplesmente verifica se o armazém de destino coincide com o destino final do pacote, caso positivo, marca o pacote como “entregue”; caso negativo, empilha-o na seção correta do armazém de chegada. Todas essas operações são feitas por meio de acessos diretos a variáveis e estruturas já existentes, sem alocação de memória extra, o que garante tempo de execução constante, $O(1)$, e complexidade espacial também $O(1)$.

- **Execução do eventos de transporte**

A execução do evento de transporte no método '*execute_transport_event*' começa removendo todos os p pacotes da seção de origem para uma pilha auxiliar, como cada remoção e empilhamento custa $O(1)$, essa etapa totaliza $O(p)$.

Em seguida, dos p pacotes, apenas os primeiros k (com $k < p$, gerando contenção) da pilha auxiliar são enviados, o que exige inserir k eventos de chegada no *MinHeap*; como cada inserção custa $O(\log(E))$ (sendo $E = p+2(n-1)$ o número máximo de eventos possíveis no *MinHeap*), esse passo tem custo $O(k \log(E))$.

Por fim, os $p-k$ pacotes restantes são reenfileirados de volta na seção em tempo constante por pacote, ou seja, $O(p-k)$. Somando todas as fases, o tempo total é:

$$O(p + k \log(E) + (p-k)) = O(p + k \log(E))$$

e, dado que k cresce em ordem menor que p , o termo $O(p)$ domina, resultando em complexidade de tempo $O(p)$. A única estrutura adicional alocada é a pilha auxiliar de até p pacotes, conferindo complexidade espacial $O(p)$.

Com a complexidade da execução de eventos definida, agora podemos determinar a complexidade de toda a simulação. Considerando o pior caso do Sistema de Escalonamento Logístico, é possível verificar que cada transporte deverá ser executado p/k vezes para

garantir a chegada de cada pacote. Como para chegar ao destino final será necessário $n-1$ transportes, temos então serão necessários $p/k * n-1$ eventos de transporte.

Quanto ao número de eventos de chegada cada um dos p pacotes percorre $n-1$ saltos, gerando exatamente n chegadas (incluindo a do armazém de origem), totalizando $n*p$ ocorrências ao longo da simulação.

Além do custo intrínseco de cada evento, é preciso extraí-lo do *MinHeap*, operação que custa $O(\log E)$. Aqui, E é o número máximo de itens no heap, ou seja, quando todos os p eventos de chegada iniciais mais os $2(n-1)$ eventos de transporte (ida e volta em cada um dos $n-1$ saltos), que resulta em $E=p+2(n-1)$.

Dessa forma, o tempo total para processar qualquer evento é a soma de seu custo de execução ($O(1)$ para chegadas ou $O(p)$ para transportes) e do $O(\log E)$ necessário para escolhê-lo no heap.

Concluimos assim que a complexidade de tempo da simulação de eventos é:

$$\begin{aligned} &O((p * n * O(\log(E)+O(1))) + (p/k * (n-1) * O(p + \log(E))) \\ &= O(p * n * \log(E) + p/k * (n-1) * p) \\ &= O(p^2/k * n-1) \end{aligned}$$

4. Estratégias de Robustez

A implementação do simulador adota várias estratégias que garantem robustez e confiabilidade ao código. Dentre elas, destacam-se as verificações de erro no acesso a arquivos, com mensagens claras quando o nome do arquivo de entrada não é fornecido ou quando há erros na abertura do arquivo

Além disso, todas as estruturas de dados fundamentais, como listas, pilhas, filas etc., realizam checagens antes de qualquer acesso ou exclusão; se a operação for inválida (por exemplo, índice fora de alcance ou estrutura vazia), ela é ignorada e uma mensagem de erro é emitida.

Durante etapas críticas da simulação, como o armazenamento e a remoção de pacotes, o sistema confirma a existência prévia do pacote e da seção correspondente para assegurar o funcionamento correto; qualquer inconsistência também resulta em aviso de erro.

A implementação também contou com um gerenciamento cuidadoso da memória, que foi possibilitado a partir da utilização de ferramentas, como o Valgrind, empregadas para detectar possíveis vazamentos de memória.

5. Análise Experimental

Para avaliar o comportamento do Sistema de Escalonamento Logístico em cenários variados, realizamos uma série de experimentos nos quais alteramos isoladamente três dimensões: número de armazéns, número de pacotes e grau de contenção no transporte. Essas variações afetam tanto o custo computacional da simulação quanto o tempo de entrega de cada pacote. Como referência, definimos um cenário base com as seguintes configurações: 500 pacotes distribuídos entre 20 armazéns; capacidade de transporte de 10 pacotes por viagem, com custo de 20 unidades; intervalo fixo de 80 unidades de tempo entre transportes; custo de remoção de 3 unidades; e frequência de postagem tal que o intervalo máximo entre pacotes

seja 10 unidades de tempo. A partir desse caso base, modificamos uma dimensão por vez para quantificar seu impacto no desempenho e nos indicadores de contenção (como o número de rearmazenamentos) do simulador.

5.1. Número de armazéns

Para verificar a influência do número de armazéns variamos o número de armazéns entre os valores 10, 20, 40, 60, 80 e mantivemos inalteradas as demais características do caso base. A figura 2 apresenta um gráfico dos resultados obtidos pela variação da quantidade dos armazéns.

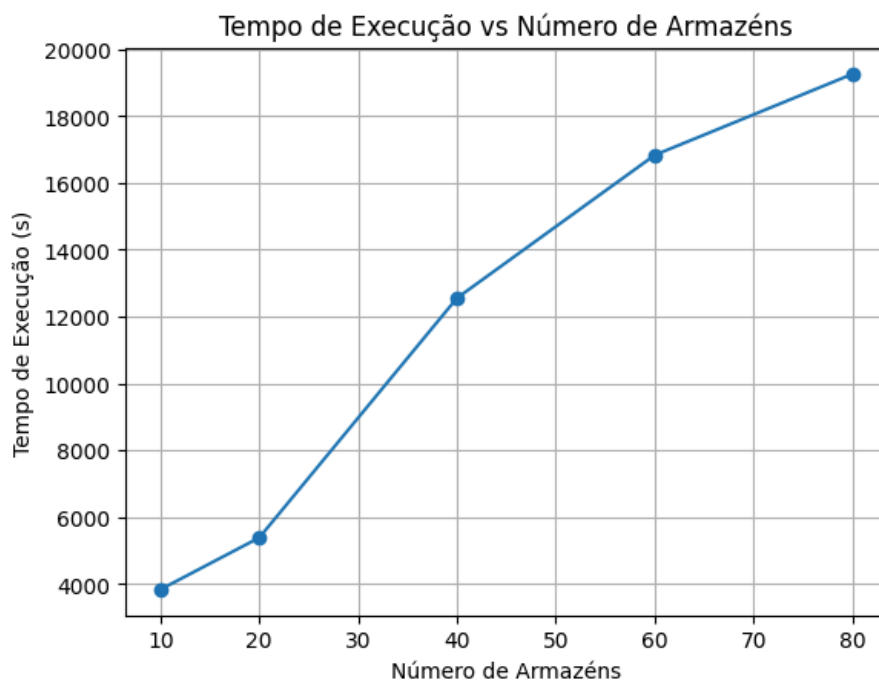


Figura 2: Influência do número de armazéns no tempo de execução

Este resultado pode ser explicado pelo fato de o número de armazéns ter um impacto direto no custo computacional do Sistema de Escalonamento Logístico, uma vez que a rota de cada pacote é pautada nestes armazéns, de modo que o aumento do número de armazéns influenciam o tamanho da rota. O aumento da rota por sua vez ocasiona uma maior quantidade de execução de eventos de transporte e de chegada em armazéns que influencia diretamente no tempo de execução.

5.2. Número de Pacotes

O segundo experimento teve como objetivo verificar a influência do número de pacotes no tempo de execução da simulação. Novamente, mantivemos as demais características da simulação como no caso base e alteramos o número de pacotes entre os valores 200, 500, 1000, 2000 e 3000. A Figura 3 mostra como o tempo de execução da simulação evolui conforme aumenta a carga de pacotes.

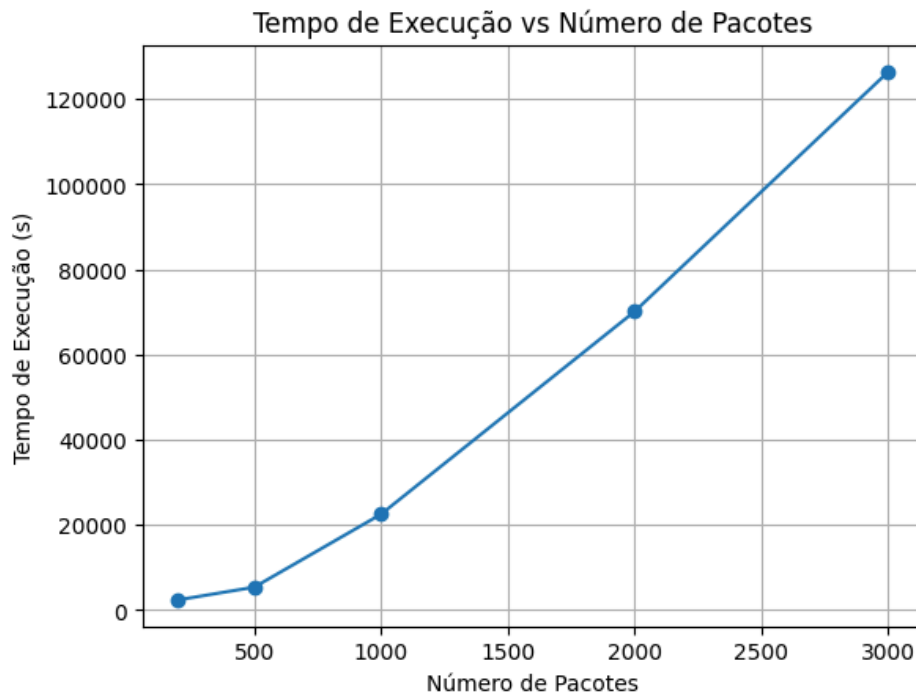


Figura 3: Influência do número de armazéns no tempo de execução

O aumento no tempo de execução é explicado pelo fato de que o número de pacotes representa a carga de trabalho processada pelo simulador: cada pacote gera eventos de transporte e chegada, de modo que uma maior quantidade de pacotes eleva tanto o número total de eventos quanto o custo computacional para processá-los.

5.3. Contenção por Transporte

Para explorar o impacto da contenção por transporte no desempenho do simulador de entregas, este experimento variou os parâmetros que mais promovem rearmazenamentos de pacotes: capacidade de transporte, latência de transporte, intervalo entre envios e frequência de chegada de pacotes.

Para avaliar separadamente o efeito de cada fator sobre o tempo de execução, definiram-se três níveis de contenção, baixa, média e alta, mantendo inalteradas todas as outras configurações do caso base. A criação desses cenários baseou-se na variação da capacidade de transporte, da latência, do intervalo entre transportes e da frequência de postagem de pacotes, sendo a capacidade de transporte e a frequência de chegada os parâmetros que mais influenciaram o tempo total de simulação.

Como métrica de contenção, contabilizou-se o número total de rearmazenamentos ocorridos durante a execução. Um volume elevado de pacotes reenviados sinaliza maior contenção por transporte, enquanto um número reduzido indica contenção menor, refletindo diretamente no custo computacional do simulador. Sendo assim, obtivemos resultados de tempo de execução para os seguintes valores de rearmazenamentos: 0, 11, 640, 1678 e 8480.

A figura 4 a seguir apresenta a influência do número de rearmazenamentos no custo computacional da simulação

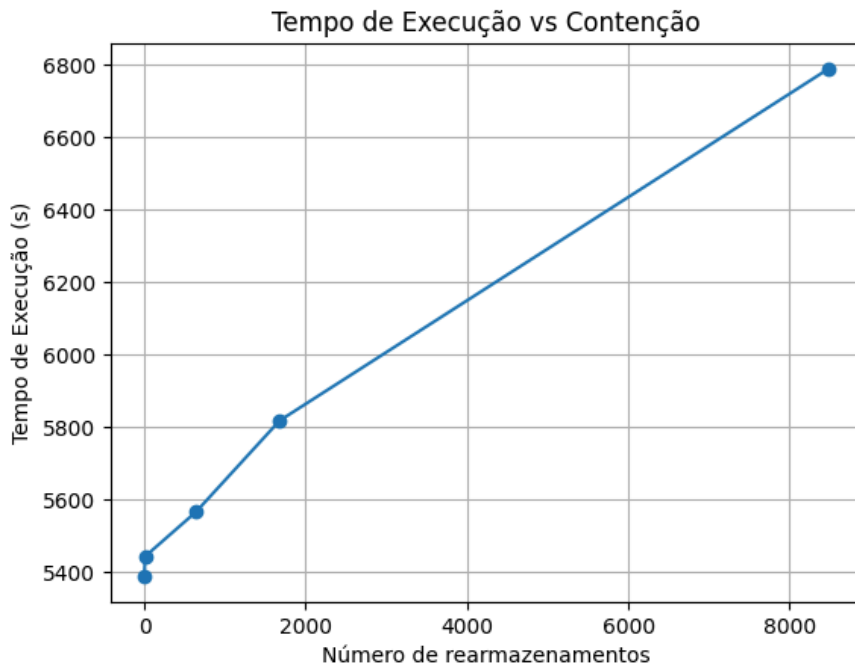


Figura 3: Influência da contenção dos transportes no tempo de execução

O resultado se justifica pelo fato de em cenários de alta contenção, quando a capacidade de transporte é insuficiente ou a frequência de chegada de pacotes é elevada, ocorrem múltiplos rearmazenamentos, gerando diversos eventos de transporte adicionais. Assim, quanto maior o número de eventos de transporte, maior o tempo total de execução da simulação.

5.4. Conclusão

Os experimentos confirmam que o custo computacional da simulação cresce de forma sensível ao aumento de cada dimensão analisada. Mais armazéns alongam as rotas e elevam o número de eventos de transporte e chegada; mais pacotes aumentam diretamente a carga de trabalho, gerando um maior volume de eventos; e maior contenção (mais rearmazenamentos) multiplica ainda mais os eventos de transporte. Esses resultados demonstram a importância dos parâmetros como capacidade, intervalo de transporte e frequência de postagem na simulação.

6. Conclusão

Neste trabalho prático foi desenvolvido um Sistema de Escalonamento Logístico, que através de um simulador de eventos discretos que cria e executa eventos dinamicamente automatiza o roteamento e o acompanhamento de pacotes numa rede de armazéns conectados por um grafo não direcionado

A execução deste trabalho prático trouxe aprendizados valiosos, entre eles, a aplicação prática da simulação de eventos discretos, técnica até então inédita, mas que se revelou extremamente eficaz para modelar e analisar sistemas complexos

Além disso, este projeto me permitiu utilizar diversas estruturas de dados vistas em teoria durante as aulas como: listas, pilhas, filas, heap e grafo. A implementação do Sistema de

Escalonamento Logístico me permitiu não apenas executá-las, mas também utilizá-las em conjunto de maneira eficiente, a fim de solucionar um problema.

Este trabalho prático evidenciou também de forma clara a influência dos parâmetros de simulação sobre o tempo de execução: ao variar isoladamente o número de armazéns, a quantidade de pacotes e os atributos de transporte, foi possível observar que cada ajuste gera impactos mensuráveis na carga de eventos processados pelo simulador.

7. Bibliografia

[1] A. Lacerda e W. Meira Jr., Slides virtuais da disciplina de Estruturas de Dados, 2020. Disponível via Moodle.

[2] T. H. Cormen, C. E. Leiserson e R. L. Rivest, Algoritmos. 3ª ed., 2017.

[3] Wikipedia contributors. Simulação de eventos discretos. Disponível em: https://pt.wikipedia.org/wiki/Simula%C3%A7%C3%A3o_de_eventos_discretos

[4]IME.USP. Busca em largura. Disponível em: https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/bfs.html

[5]IME.USP. Algoritmo de Dijkstra. Disponível em: https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/dijkstra.html

8. Proposta de Aprimoramento Extra: Custo de Transporte Variável

O Sistema de Escalonamento Logístico partia da premissa de que todos os transportes entre armazéns tinham custo idêntico. Na prática, porém, cada trajeto exige tempos de deslocamento distintos, e essa uniformidade acabava distanciando a simulação da realidade. Para resolver isso, passou-se a ler a topologia como um grafo cujas arestas são ponderadas pelo tempo estimado de viagem entre armazéns, de modo que o custo de uma rota não dependa apenas do número de saltos, mas também da soma dos pesos ao longo do caminho. Para encontrar, de forma eficiente, o trajeto de menor custo total, foi adotado o algoritmo de Dijkstra. Ele inicia atribuindo distância zero ao vértice de origem e “infinita” a todos os demais, e então, em cada iteração, extrai da fila de prioridade o nó com menor distância acumulada, atualiza os valores de seus vizinhos (relaxamento de arestas) e repete até alcançar o destino. Esse procedimento garante a seleção de rotas que minimizam o tempo de transporte, aproximando significativamente nossa simulação das condições operacionais reais.

Para validar essa melhoria que a mudança de algoritmo para cálculo da rota gera, foram conduzidos experimentos que comparam diretamente as rotas geradas pela busca em largura e pelo algoritmo de Dijkstra. Em cada teste, simulamos a entrega de quatro pacotes e registramos o custo total de suas rotas.

As figuras a seguir mostram, lado a lado, o custo de percurso obtido em cada abordagem, evidenciando como o uso de pesos nas arestas pelo Dijkstra leva a trajetos mais eficientes.

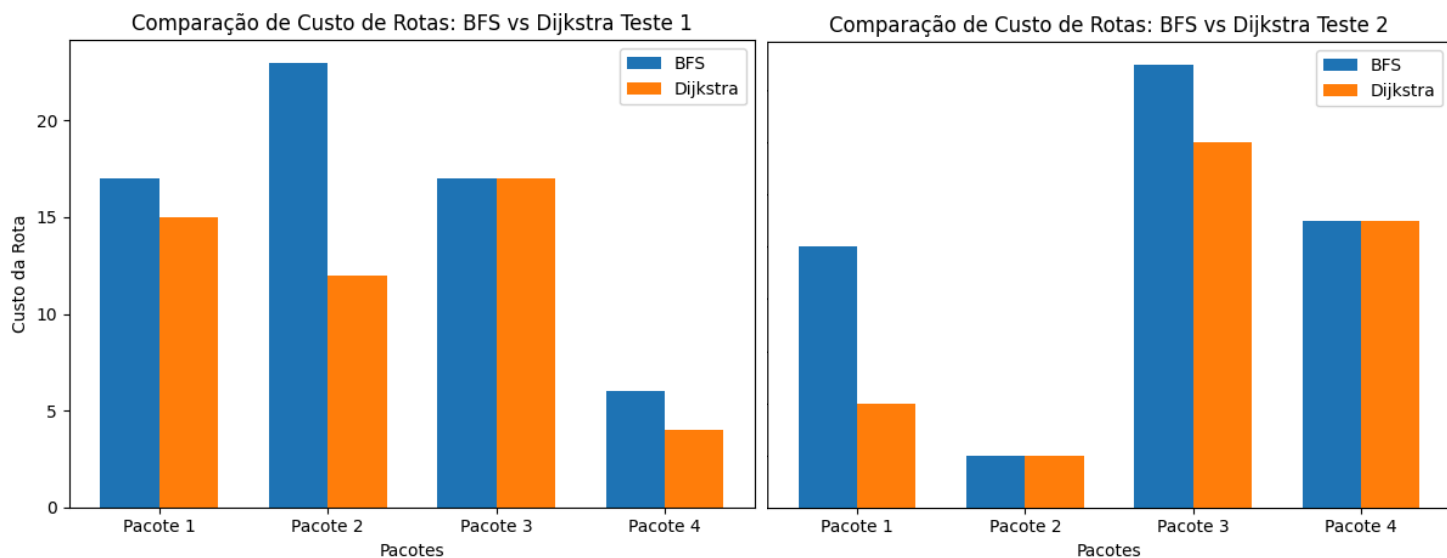


Figura 4: resultado de custo das rotas dos teste 1 e 2

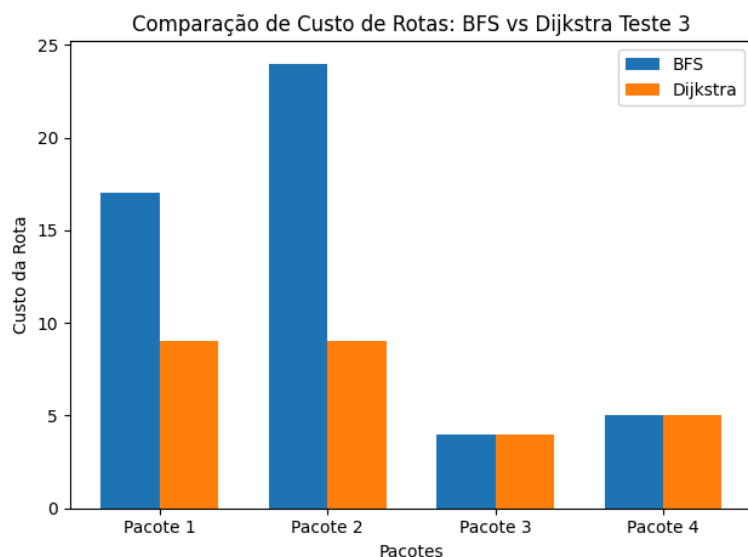


Figura 5: resultado de custo das rotas do teste 3

Em todos os testes (que estão disponíveis na pasta ‘Tests’ no .zip da implementação extra) as rotas utilizando o algoritmo de Dijkstra tiveram custo menor ou igual a busca em largura, o que demonstra a vantagem de em simulações mais realista utilizar Dijkstra para garantir um transporte mais eficiente.